

Field Validation Protocol: Satellite Terrain Masking

Objective: To validate if the Python Simulation accurately predicts the specific time windows when terrain (mountains/ridges) blocks or reveals satellite connectivity.

Prerequisites:

- **Hardware:** One satellite-enabled device (iPhone, Garmin, or Starlink).
- **Software:** The "satellite_los_analysis-constell" Python Notebook.
(https://github.com/ccommiss-dot/satellite_los_analysis-constell)
- **Data:** A .tif Digital Elevation Model (DEM) of the test region.
(<https://portal.opentopography.org/datasets>)

Phase 1: Digital Preparation (At Home)

1. Select Network & Device

Before running any code, determine which Constellation TLE (Two-Line Element) to load based on your hardware.

- **iPhone 14+:** Select **Globalstar** for Apple
- **Garmin inReach / Zoleo:** Select **Iridium**
- **Starlink:** Select **Starlink**
- **Spot:** Select **Globalstar**
- **T-Mobile:** Select **Starlink v2**

2. Acquire Elevation Data (DEM)

1. Navigate to a DEM database (e.g., OpenTopography or SRTM Downloader).
2. Select the region where you intend to hike.
3. **Critical Buffer:** Ensure the downloaded .tif file covers your hike location **PLUS a 10-mile radius** in every direction.
 - *Why?* To accurately calculate ray tracing, the code needs to see the mountains *distant* from you that might block the horizon, not just the ground immediately beneath your feet.

3. Site Selection & Simulation

1. Load the .tif into the Notebook (Cell 2-AUTO).
2. Run the Notebook to generate the **Topographical Map**.

3. Visually select **3 Test Sites (A, B, C)** on the map along the hiking path you plan to use where you expect terrain to interfere (e.g., deep canyons, north-facing slopes, or ridge shadows).
4. You can select one specific GPS coordinate as your observer location, this is selected in Cell 5.
5. You can select a specific time (in UTC) for when you want to calculate, meaning you can make retrospective analysis, this is selected in Cell 4
6. Run the simulation cells **in order**.
7. Generate the **Output Timelapse Map** (Cell 19).

Phase 2: Route Planning (The "Transition" Search)

Do not just hike randomly. You are looking for a **Variance Time**—a specific moment when the simulation predicts the connection will change status.

1. **Analyze the Timelapse:** Watch the connection status for your Site A, B, and C or your observer pin over a 24-hour period.
2. **Identify the Transition:** Find the exact minute the map changes color (e.g., Green to Red or Red to Green).
 - *Example:* "At Site B (The Canyon), connection is lost at 10:15 UTC."
3. **Plan the Hike:** Schedule your hike so you arrive physically at Site B exactly at the transition time (10:15 UTC).

Phase 3: Field Execution (The Test)

Environmental Constraint: For valid results, you should choose test spots **clear of local clutter**, the data works best with alpine zones or deserts.

- **AVOID:** Dense tree canopies, tall buildings, power poles, or standing right next to a vehicle.
- **SEEK:** Open clearings, large rocks, or trail intersections where only the *terrain* (mountains/earth) obscures the sky.

The "10-Minute Window" Protocol

Perform this sequence for every "Transition Time" you identified in Phase 2.

Step 1: The Pre-Check (T-Minus 10 Minutes)

- **Action:** 10 minutes *before* the predicted change (e.g., 10:05 UTC).
- **Log Location:** Record exact GPS coordinates.

- **Test:** Attempt to connect (Satellite Demo or Message Check).
- **Expected Result:** Should match the "Before" state of the simulation.

Step 2: The Transition (T-Zero)

- **Action:** At the exact predicted minute (e.g., 10:15 UTC).
- **Log Location:** Confirm you are at the target coordinates.
- **Test:** Attempt to connect.
- **Observation:** This is the critical test. Does the signal drop (or appear) exactly when the satellite sets behind (or rises above) the ridge?

Step 3: The Post-Check (T-Plus 10 Minutes)

- **Action:** 10 minutes *after* the predicted change (e.g., 10:25 UTC).
- **Test:** Attempt to connect.
- **Expected Result:** Should match the "After" state of the simulation.

Phase 4: Data Logging & Comparison

Use this template to record your findings. **All times must be logged in UTC** to match the Python tool.

Site	GPS Lat/Lon	Phase	Time (UTC)	Sim Prediction	Real World Result	Variance?
Site A	34.23, -117.8	T-10	14:00	Connected	Connected	No
Site A	34.23, -117.8	T-0	14:10	Disconnects	Lost Signal	Match
Site A	34.23, -117.8	T+10	14:20	No Signal	No Signal	No

Troubleshooting Results

- **Sim says "Connected" but in the Real World "No Signal":**
 - Did you have a clear view? (Trees?)
 - Was your body blocking the device? (Did you spin to face the satellite?)
 - Is the DEM resolution too low (missed a small cliff)?
- **Sim says "No Signal" but in the Real World "Connected":**
 - Did the signal "bend" (diffraction) over the peak?
 - Was the satellite higher in the sky than the TLE predicted?

Common Data Errors (Troubleshooting)

- **Error 1: The "Narrow Strip" (Insufficient Coverage)**
 - **Problem:** The tool reports a connection, but a distant mountain is actually blocking the view.
 - **Cause:** The downloaded DEM file follows the trail too closely. The ray-tracing algorithm runs off the edge of the map and assumes the world is flat beyond that point.
 - **Fix:** Always download a square box covering your hike location **plus a 10-15 mile buffer** in every direction. If you can see a mountain with your eyes, it must be inside your `.tif` file.
- **Error 2: The "Swiss Cheese" (Data Voids)**
 - **Problem:** The tool shows a "Connected" line passing straight through a solid mountain.
 - **Cause:** Raw satellite elevation data often has missing pixels (holes) due to shadows or reflections. The code interprets these empty pixels (`-9999`) as holes in the earth, allowing the signal to pass through.
 - **Fix:** Try to use datasets labeled **"SRTM Void Filled"** or **"NASADEM"**. These have been mathematically patched to remove holes.